

Question 1: Can you describe how you would implement authentication and authorization in a spring boot application?

Overview

Authentication verifies who a user is. Authorization decides what that user can do. In Spring Boot you implement both using Spring Security (plus optional token/OAuth layers) to protect endpoints, validate credentials, and enforce roles/permissions [GeeksForGeeks](#) [Spring](#).

High-level approach (easy-to-remember checklist)

1. Add dependencies: **spring-boot-starter-security**, optionally **spring-boot-starter-oauth2-client** or JWT library (jjwt / java-jwt)
2. Define users and roles (database table or external provider)
3. Implement credential checks (UserDetailsService + PasswordEncoder)
4. Wire SecurityFilterChain to secure routes and apply auth rules
5. Use JWT or OAuth2 for stateless APIs; session/cookie for server-side apps
6. Protect methods or endpoints with roles/authority annotations (PreAuthorize / hasRole)
7. Test and monitor (unit + integration + logs)

Mnemonic: “D-FU-S-M-T” = Dependencies, Foundation (users), UserDetails, SecurityChain, Method rules, Test.

Concrete implementation steps with a minimal code sketch

1. Dependencies

- Add spring-boot-starter-security; add JWT or OAuth client if you need token-based/OAuth flows [GeeksForGeeks](#).

2. User representation and persistence

- Create User and Role entities, store users in a database (JPA repository) or use an external IdP.

3. UserDetailsService and PasswordEncoder

- Implement UserDetailsService that loads UserDetails from DB. Use BCryptPasswordEncoder to hash passwords.

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

@Service
public class DbUserDetailsService implements UserDetailsService {
    private final UserRepository repo;
    public DbUserDetailsService(UserRepository repo) { this.repo = repo; }
    @Override
```

```

public UserDetails loadUserByUsername(String username) {
    User user = repo.findByUsername(username)
        .orElseThrow(() -> new UsernameNotFoundException(username));
    return new org.springframework.security.core.userdetails.User(
        user.getUsername(),
        user.getPassword(),
        user.getRoles().stream().map(r -> new SimpleGrantedAuthority("ROLE_"
+ r.getName()))).toList()
    );
}
}

```

4. SecurityFilterChain configuration (Spring Boot 3+ style)

- Configure which endpoints need auth, what roles are allowed, and add token filters for JWT if used.

```

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
    http
        .csrf().disable()
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/public/**").permitAll()
            .requestMatchers("/api/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        )
        .sessionManagement(sess ->
            sess.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .httpBasic(); // or .oauth2ResourceServer().jwt() for JWT/OAuth
    return http.build();
}

```

5. JWT flow (stateless APIs)

- On login, validate credentials, issue signed JWT with claims and roles. Add a filter that validates JWT on each request and sets Authentication in SecurityContext.

6. OAuth2 / OIDC option

- For social or enterprise logins, wire spring-boot oauth2 client or resource-server support to delegate auth to providers like Google, GitHub, or an IdP [Spring](#).

7. Method-level security

- Enable and use annotations for fine-grained control: `@EnableMethodSecurity` and then `@PreAuthorize("hasRole('ADMIN')")` on service methods.

8. Testing and hardening

- Test happy and unhappy paths; rotate keys, use short JWT lifetimes, refresh tokens, enforce strong password policies, log auth events, and rate-limit login endpoints.

Quick mental model to remember

- Authentication = “prove your identity” (login, token exchange).
- Authorization = “what you can access” (roles, scopes, ACLs).
- Remember: UserDetailsService + PasswordEncoder = credential check;
SecurityFilterChain = route protection; JWT/OAuth = stateless, session = stateful.

Common choices and when to use them

- Small internal app: session-based auth with BCrypt + role checks.
- Public REST API: JWT signed tokens, stateless, short TTL + refresh tokens.
- Single Sign-On / corporate: OAuth2 / OIDC with a trusted IdP [Spring](#).
- Microservices: central auth server (OAuth2) or token verification in each service.